

LAB 2 · WEEKS 3–4

Secure Isolation & Multi-Tenancy

Compute, network and storage isolation — Docker & Kubernetes

Lab Learning Outcomes

At the end of this lab, you will be able to:

1. Demonstrate **compute isolation** by separating tenants into containers and Kubernetes namespaces.
2. Observe the **default-open** behaviour of shared infrastructure and explain why it is a risk.
3. Implement **network isolation** with a default-deny NetworkPolicy and prove cross-tenant traffic is blocked.
4. Enforce **storage isolation** so one tenant cannot read another tenant's data or secrets.
5. Explain **data remanence** and demonstrate secure deletion.

Course & Assessment Mapping

Item	Mapping
Course Learning Outcome	CLO2 — Construct secure cloud operations that safeguard data integrity
Lecture topics	Week 3 (Secure Isolation of Physical & Logical Infrastructure)
Value / skill clusters	VBE3 (Integrity) · SC8 (Integrated Problem-Solving)
Assessment	Lab report (screenshots + CLI output + short answers) — contributes to the Lab Assignment

Lab Arrangement (2 Sessions over 2 Weeks)

Session	Week	Focus
Session A	Week 3	Compute isolation: containers, namespaces, resource quotas, and the default-open risk (Tasks 1–3)
Session B	Week 4	Network & storage isolation: default-deny NetworkPolicy, per-tenant secrets, data remanence (Tasks 4–6), then the report

Note: Session A shows the problem (shared, open infrastructure). Session B applies the controls that make it safely separated. Keep outputs from both weeks for the report.

Technical Prerequisites

- A laptop with **at least 8 GB RAM** and admin rights.
- **Docker Desktop / Docker Engine** — free.
- **kind** and **kubectrl** — free.
- Internet only for the first image download; the lab then runs offline.

yaml is indent sensitive

Security tip: This lab uses a Kubernetes cluster with a CNI that enforces NetworkPolicy. We use kind with Calico so that the isolation rules actually take effect (the default kind network does not enforce policies).

Session A (Week 3) — Compute Isolation & the Default-Open Risk

Setup — Cluster with Policy Enforcement

the network security/enforcement layer for Kubernetes.

```
# Create a cluster with the default CNI disabled, then install Calico
cat <<EOF | kind create cluster --name ccse-lab2 --config=-
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
networking:
  disableDefaultCNI: true
  podSubnet: 192.168.0.0/16
EOF

kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/calico.yaml
kubectl -n kube-system rollout status daemonset/calico-node --timeout=180s
```

Container Network Interface

Kubernetes cluster = apartment building
Pods = people/rooms
Namespaces = different tenants/companies
NetworkPolicy = rules saying who is allowed to communicate with whom
Calico = firewall that actually enforces those rules

install Calico + check Calico is ready

Note: If you have no internet in the lab room, your instructor can provide the calico.yaml file locally — apply it with `kubectl apply -f calico.yaml`.

Task 1 — Two Tenants on One Cluster

Model two customers as two namespaces sharing the same physical infrastructure.

```
kubectl create namespace tenant-a
kubectl create namespace tenant-b

# Deploy a simple web server for each tenant
kubectl -n tenant-a create deployment web --image=nginx
kubectl -n tenant-b create deployment web --image=nginx
kubectl -n tenant-a expose deployment web --port=80
kubectl -n tenant-b expose deployment web --port=80
kubectl get pods,svc -n tenant-a
```

Create a web application for Tenant A & B

Expose Tenant A & B web server

Check what is running Show me the Pods and Services currently inside tenant-a.

Task 2 — Observe the Default-Open Risk

By default, pods in one namespace **can reach** pods in another. Prove it: launch a test pod in tenant-a and connect to tenant-b's service.

```
# Get tenant-b's service IP
kubectl get svc web -n tenant-b -o jsonpath='{.spec.clusterIP}'; echo

# From tenant-a, curl tenant-b (replace <B_IP>)
kubectl -n tenant-a run probe --rm -it --image=curlimages/curl --restart=Never \
  -- curl -s -m 5 http://<B_IP> -o /dev/null -w 'HTTP %{http_code}\n'
```

Caution: A result of HTTP 200 means tenant-a reached tenant-b. On shared infrastructure, isolation is NOT automatic — you must configure it. This is the multi-tenancy risk from Week 3.

Task 3 — Contain the Noisy Neighbour (Resource Quotas)

Isolation is also about resources. Apply a quota so one tenant cannot exhaust the shared node.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-a-quota
  namespace: tenant-a
spec:
  hard:
    requests.cpu: "1"
    requests.memory: 512Mi
    pods: "5"
EOF
kubectl describe resourcequota tenant-a-quota -n tenant-a
```

ResourceQuota is to limit how much shared Kubernetes resources one tenant can consume.

Multi-tenancy isolation isn't only about stopping Tenant A from accessing Tenant B.

We also need resource isolation so one tenant cannot exhaust shared infrastructure and affect other tenants.

Note: End of Session A. Save the HTTP 200 result — you will show that the SAME probe returns a failure after applying network policy in Session B.

Session B (Week 4) — Network & Storage Isolation

Task 4 — Default-Deny Network Isolation

Apply a default-deny ingress policy to each tenant, then allow only same-namespace traffic. This is the segmentation principle: deny by default, permit by exception.

```
# Deny ALL ingress into tenant-b
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: tenant-b
spec:
  podSelector: {}
  policyTypes: [Ingress]
EOF

# Re-run the SAME probe from Task 2 – it should now TIME OUT / fail
kubectl -n tenant-a run probe --rm -it --image=curlimages/curl --restart=Never \
  -- curl -s -m 5 http://<B_IP> -o /dev/null -w 'HTTP %{http_code}\n'
```

Security tip: Capture both results side by side: HTTP 200 (before) and timeout (after). This single before/after is the strongest evidence of enforced network isolation.

Task 5 — Storage & Secret Isolation

Each tenant stores a secret. Prove that tenant-a cannot read tenant-b's secret — storage isolation enforced by RBAC.

```
# Create a secret in each tenant
kubectl -n tenant-a create secret generic data --from-literal=value=SECRET_A
kubectl -n tenant-b create secret generic data --from-literal=value=SECRET_B

# A service account scoped to tenant-a only
kubectl -n tenant-a create serviceaccount app-a
kubectl -n tenant-a create role reader --verb=get --resource=secrets
kubectl -n tenant-a create rolebinding rb --role=reader --serviceaccount=tenant-a:app-a

SA=system:serviceaccount:tenant-a:app-a
kubectl auth can-i get secrets -n tenant-a --as=$SA # expect: yes
kubectl auth can-i get secrets -n tenant-b --as=$SA # expect: no
```

Task 6 — Data Remanence & Secure Deletion

When data is 'deleted', is it really gone? Demonstrate remanence and a secure wipe inside a container volume.

```
# Create a file, delete it normally, then show the bytes may persist
```

```
docker run --rm -v ccse-vol:/data alpine sh -c \  
  'echo SENSITIVE-PATIENT-RECORD > /data/phi.txt; sync; rm /data/phi.txt; \  
  grep -a SENSITIVE /data/* 2>/dev/null; echo scan-done'  
  
# Secure wipe: overwrite before delete (shred)  
docker run --rm -v ccse-vol:/data alpine sh -c \  
  'echo SENSITIVE > /data/phi2.txt; sync; \  
  dd if=/dev/zero of=/data/phi2.txt bs=1k count=1 conv=notrunc; rm /data/phi2.txt;  
  echo wiped'
```

Note: In cloud storage you rarely control physical blocks, so the practical answer to remanence is cryptographic erasure (destroy the key). You will do exactly that in Lab 3.

Deliverables & Assessment

1. Screenshots (label each clearly)

- Pods/services running in tenant-a and tenant-b (Task 1).
- The **before** probe result: HTTP 200 across tenants (Task 2).
- The **after** probe result: timeout once the NetworkPolicy is applied (Task 4).
- The two `auth can-i` results proving secret isolation (Task 5).
- The remanence scan and the secure-wipe output (Task 6).

2. Short-Answer Questions

- Q1.** Why can containers in different namespaces reach each other by default, and why is that dangerous in multi-tenant cloud?
- Q2.** Explain the **default-deny** principle and how your NetworkPolicy implements it.
- Q3.** How do virtual machines and containers differ in isolation strength? When would you add a VM boundary?
- Q4.** What is **data remanence**, and why is cryptographic erasure the preferred cloud solution?
- Q5.** Which of the three isolation dimensions (compute, network, storage) did each task exercise?

3. Verification Command

```
kubectl get networkpolicy -A
kubectl describe resourcequota tenant-a-quota -n tenant-a
```

Security Best-Practices Checklist

- ☐ Tenants are separated into distinct namespaces.
- ☐ A default-deny NetworkPolicy blocks cross-tenant traffic (verified before/after).
- ☐ Resource quotas prevent a noisy-neighbour from exhausting shared capacity.
- ☐ Per-tenant secrets are unreadable by other tenants (RBAC enforced).
- ☐ Secure deletion / cryptographic erasure is understood for data remanence.

Cleanup & Teardown

```
kind delete cluster --name ccse-lab2
docker volume rm ccse-vol
```

Expansion Ideas (Advanced Students)

- Add an **egress** default-deny policy and allow only DNS + a specific service (micro-segmentation).
- Enforce **Pod Security Standards** (restricted) so no tenant can run privileged pods.
- Try a runtime sandbox such as **gVisor** to strengthen compute isolation beyond the shared kernel.

- Use **Calico GlobalNetworkPolicy** to enforce tenant isolation cluster-wide, not per namespace.

References

- Course lecture — Week 3 (Secure Isolation of Physical & Logical Infrastructure).
- Kubernetes Network Policies — kubernetes.io/docs/concepts/services-networking/network-policies
- Calico documentation — docs.tigera.io
- CSA Security Guidance v5 — Infrastructure & Networking domain.